Matrice de couverture des fixtures - SCE 2.0 Reports v1.3

Date : 2026-05-14 **Test pilote :** `crates/pos-fiscal/tests/reports_fixtures.rs` **Fixtures source :** `docs/20260821_UC_Reports_1.3/`

Révision du jeu publié : le dossier `docs/20260821_UC_Reports_1.3/` reprend la version 1.3 (le changelog ne change pas) ; les fichiers de fixtures ont été republiés avec des horodatages portant les millisecondes (`2024-07-30T13:00:35.000+02:00`).

Reports v1.3 ajoute la suite de rapports X/Z au corpus de certification. Cette phase valide que chaque réponse publiée se désérialise dans notre `SignResult` et que la dérive de schéma v1.3 (suppression du `reportBookingDate` des rapports X) est reflétée dans notre `inputs.rs`.

Matrice de couverture

UC	Événement	Test	Résultat
M180_signReportTurnoverX	R (TurnoverX)	`m180_turnover_x_response_parses`	✓
M181_signReportTurnoverZ	R (TurnoverZ)	`m181_turnover_z_response_parses`	✓
M182_signReportUserX	R (UserX)	`m182_user_x_response_parses`	✓
M183_signReportUserZ	R (UserZ)	`m183_user_z_response_parses`	✓
M184_signReportTurnoverZUserZ	R (TurnoverZ + UserZ)	`m184_combined_z_response_parses_and_counters_incremen`	✓
InputReport / T1-T8 (événements de staging)	N + P	`input_report_staging_fixtures_parse` (8 tests)	✓
v1.3 - X reports omettent `reportBookingDate`	-	`x_reports_omit_report_booking_date`	✓
v1.3 - Z reports incluent `reportBookingDate`	-	`z_reports_include_report_booking_date`	✓

Total : 8 tests, tous passants.

Combiné avec `uc_fixtures.rs` (Basic Set v1.7), le label d'événement **R** est désormais exercé, complétant le set des 8 lettres : N, P, F, S, I, C, T, R.

M184 : émission Z combinée

Le Z de fin de journée mandaté par la spec est une seule opération GraphQL aliasant à la fois `signReportTurnoverZ` et `signReportUserZ` :

```
mutation M184_signReportTurnoverZUserZ(
  $turnover: ReportTurnoverZInput!
  $user: ReportUserZInput!
  $training: Boolean!
) {
  signReportTurnoverZ(data: $turnover, isTraining: $training) { ...resultFields }
  signReportUserZ(data: $user, isTraining: $training)         { ...resultFields }
}
```

La réponse fixture porte deux payloads `SignResult` dans une seule enveloppe. Le test assert :

- les deux se désérialisent en `SignResult`,
- les deux sont du label d'événement R,
- le `signReportUserZ.fdmRef.totalCounter` est égal à `signReportTurnoverZ.fdmRef.totalCounter + 1` - prouve que le FDM a signé les deux événements séquentiellement dans la même opération atomique,

- les deux réponses référencent le même `fdmId` (un seul FDM dans le setup de test).

Dérive de schéma v1.3 - résultat d'audit

Champ	Changement de spec	Notre `inputs.rs`	Verdict du test
`ReportTurnoverXInput.reportBookingDate`	Supprimé en v1.3	Champ **absent** dans la struct	✓ correspond
`ReportUserXInput.reportBookingDate`	Supprimé en v1.3	Champ **absent** dans la struct	✓ correspond
`ReportTurnoverZInput.reportBookingDate`	Requis	`report_booking_date: String`	✓ correspond
`ReportUserZInput.reportBookingDate`	Requis	`report_booking_date: String`	✓ correspond

Aucune modification de code n'a été nécessaire pour l'audit v1.3 - les structs étaient déjà alignées. Les tests fixture attraperont maintenant toute régression qui ré-introduirait le champ sur les rapports X.

Écarts d'implémentation suivis

Les deux écarts identifiés lors de l'audit fixture sont **résolus** dans la build de soumission :

1. **Co-émission M184 - implémentée.** `crates/pos-engine/src/commands/report.rs::do_generate_z_report` émet désormais le Z canonique : `signReportTurnoverZ` + `signReportUserZ` dans **une seule opération GraphQL** (`FdmClient::sign_report_turnover_z_user_z`), avec deux numéros de ticket séquentiels et un archivage **atomique** des deux événements R (transaction SQLite : les deux archivés ou aucun). Le rapport utilisateur X (M182) dispose de sa propre commande `do_generate_user_x_report`, déclenchée depuis le menu Rapports. E2E : `spf_e2e_reports.rs::{m184_*, m182_*}`.
2. **Info device FDM dans les payloads X/Z - réelle.** Le bloc `fdmDevices` est peuplé depuis les `fdmRef` (fdmId, `totalCounter`, `fdmDateTime`) des événements archivés du rapport, premier et dernier par compteur (`report_totals::fdm_devices`), plus de placeholder. Vérifié par l'assertion `fdmDevices[0].fdmId == <identité archivée>` dans le test M180.

   Mise à jour (calcul des totaux depuis l'archive signée) : une révision antérieure de ce document attribuait les agrégats par utilisateur aux fonctions `pos_db::queries::reports::{get_user_activity, get_user_payment_totals, get_user_social_events, get_fdm_ref_range}`. Ces fonctions ont été **supprimées** (commit `184f36c`) ; `crates/pos-db/src/queries/reports.rs` ne contient plus que `next_z_report_number`, `ZReportSummary`, `list_z_reports` et `save_z_report` (numérotation et index des Z, pas de calcul de totaux). Le calcul réel est purement fonctionnel, dans `crates/pos-fiscal/src/report_totals.rs`, à partir des `fiscal_events` archivés et signés de la période d'ouverture en cours - voir la section suivante.

Calcul des totaux : `report_totals.rs`, à partir de l'archive signée

`crates/pos-fiscal/src/report_totals.rs` reconstruit chaque objet du rapport (POS v2.2 §2.2.2) en relisant les `fiscal_events` archivés de la période d'ouverture courante (table `booking_periods`, migration `003` - voir *SCHEMA_BD_FR.md* §3.5 et *STRATEGIE_COMPTEURS_FR.md* §6), jamais les tables mutables `orders`/`order_lines`/`payments` :

Objet du rapport	Calcul
`transactions`	Nombre d'événements et Σ montant par label, pour tout événement portant une `transaction`
`departments`	Σ montant des lignes des événements N, par `departmentId`/`departmentName`
`vats`	Σ du `vatCalc` renvoyé par le FDM sur chaque N (repli sur les taux standard A/B/C/D si absent)
`payments`	Les `financials` des événements N, par méthode (`id`/`name`/`type`)
`negQuantities`	Les lignes à quantité négative, par `negQuantityReason` (`CORRECTION`/`REFUND`), comptées une fois, sur l'événement qui les a transmises en premier (FAQ Q303) : l'enregistrement de commande (P, `ORDER`) pour une table, la vente N pour une vente directe ou un remboursement, jamais une addition provisoire. La vente d'une table renvoie ses corrections déjà enregistrées, même avant un Z : elles ne sont pas recomptées
`drawersOpenCount`	Événements F dont `eventOperation == DRAWER_OPEN`
`users[].totalAmount`	Σ `transaction.transactionTotal` des événements N de l'employé
Plage d'activité utilisateur	Tous les événements de l'employé **sauf** les rapports R eux-mêmes
`posDevices` / `fdmDevices`	Bornes premier/dernier par `posId`/`terminalId`, et par `fdmId`/`totalCounter`

Tous les montants JSON (multiples de 0,01) sont convertis une seule fois en centimes avec arrondi, puis sommés en entier - jamais en flottant cumulé, pour éviter la dérive à la sommation (0,01 € perdu sur des sommes comme 3,04 €, voir `report_totals.rs::amounts_are_summed_in_cents_without_float_drift`).

Les identifiants de paiement sont ceux de pili-pos (`crates/pos-fiscal/src/event.rs`) : `"1"`/`"Cash"` pour les espèces, `"2"`/`"Card"` pour la carte. Sans terminal de paiement relié, la carte est toujours `CARD_DEBIT` : l'opérateur ne distingue pas débit et crédit à l'encaissement (méthode `MANUAL`). Avec un terminal relié, le terminal rend lui-même le type de carte (méthode `AUTOMATIC`) et les rapports comptent chaque type à part : `"2"`/`"Card"` (`CARD_DEBIT`), `"6"`/`"Credit card"` (`CARD_CREDIT`) et `"7"`/`"Card (type unknown)"` (`CARD_UNKNOWN`), sans jamais mélanger deux types dans un même décompte (`crates/pos-fiscal/src/payment.rs`).

Rejeu du scénario complet InputReport (T1-T8)

En complément des fixtures M180-M184 ci-dessus (chacune un scénario isolé de désérialisation de réponse, `reports_fixtures.rs`), le service complet du scénario officiel `InputReport` (T1-T8, `docs/20260821_UC_Reports_1.3/In` est rejoué deux fois pour valider le calcul des totaux lui-même :

- **Calcul pur** - `crates/pos-fiscal/tests/report_totals_fixtures.rs` construit les 8 `ReportEvent` des fixtures T1-T8 et compare la sortie de `report_totals` aux rapports M180/M182/M184 publiés.
- **Bout en bout** - `crates/pos-engine/tests/spf_e2e_report_fixtures.rs` rejoue les 8 tickets via les vraies commandes (`do_add_order_line`, `do_sign_table_order`, `do_sign_pre_bill`, `do_checkout`, `do_refund_checkout`, `do_generate_x_report`, `do_generate_user_x_report`, `do_generate_z_report`) contre un FDM simulé qui rejoue les réponses du scénario, puis compare les rapports que pili-pos construit depuis sa propre archive aux mêmes fixtures publiées.

Les deux tests documentent un écart assumé entre le rapport publié et le rapport recalculé, avec la même conclusion : le rapport publié compte 3 tickets P pour 198 € (T1 exclu), alors que le scénario enregistre 4

événements P (T1, T5, T6, T7 - commandes et pré-addition) pour 250 € ; la spec §2.2.2 définit `ticketCount` comme le nombre d'événements de ce label dans la période, donc pili-pos rapporte 4 et 250 € - conforme au texte de la spec et à l'arrêté ministériel du 29 avril 2024 (art. 29, t : le nombre et le montant des tickets de chaque type), différent du chiffre publié dans la fixture. Aucun texte ne permet d'exclure T1 : ce choix est délibéré. Écarts additionnels documentés et volontaires : l'`id`/`name` de paiement sont ceux de pili-pos (ci-dessus), les paiements carte sont `CARD_DEBIT` sans terminal relié (fixture : `CARD_CREDIT`), les noms de département sont ceux du catalogue en français, et `posDevices`/`fdmDevices` reflètent le terminal unique et le FDM simulé du test.

Le scénario T1-T8 se déroule dans une unique période d'ouverture persistée (`booking_periods`) : aucun Z n'intervient avant la fin du scénario, donc les rapports X (M180) et X utilisateur (M182) générés en cours de service lisent la même période encore ouverte que le Z final (M184).

Reproductibilité

```
cd pili-pos
cargo test --package pos-fiscal --test reports_fixtures -- --nocapture
```

S'exécute également dans le cadre de la suite de tests workspace complète ; totaux par binaire de test dans *RAPPORT_TESTS_BASELINE_FR.md*, régénérés par `scripts/generate_test_report.py`.